Energy Efficiency of LLM-Generated Code: A Cross-Language and Cross-Platform Empirical Study

Brisild Capani
DHBW Stuttgart
inf24039@lehre.dhbw-stuttgart.de

Seyyid Bachri Oglou
DHBW Stuttgart
inf24071@lehre.dhbw-stuttgart.de

***Abstract*—Evaluations of LLM-generated code focus on correctness rather than the energy the code consumes when executed. We study energy efficiency along three factors: the generating LLM (Claude, GPT, and Gemini via their official coding CLIs), the language (Python, Java, C++), and the hardware platform. 630 programs for 14 tasks are checked for correctness and measured under controlled conditions.**

TODO: Final results after all platforms.

***Index Terms*—Large language models, code generation, energy efficiency, green software engineering, empirical study**

I. Introduction

Code-generation benchmarks measure correctness, not the energy generated code consumes. Prior studies find that this energy varies between models [1]–[3], but each varies only some factors: several languages on similar machines [1], [4], or heterogeneous platforms for Python only [3]. Whether "model A generates more efficient code than model B" holds beyond one language and machine is untested [5].

In a full factorial experiment, three LLMs, accessed through their official coding CLIs, solve 14 tasks in Python, Java, and C++, and every correct program is measured on heterogeneous platforms. We ask which LLM generates the most energy-efficient code (**RQ1**), how languages (**RQ2**) and platforms (**RQ3**) differ, whether the LLM ranking depends on language and platform (**RQ4**), and how energy, time, and memory relate (**RQ5**).

II. Related Work

Most studies measure Python code from several LLMs on one x86 machine [2], [6], [7]. Cursaru et al. [8] use one model on a Raspberry Pi; Apsan et al. [3] use three platforms, but only Python and without platform interactions; Solovyeva et al. [1] cover our three languages on two machines with C++ compiled at `-O0`. Runtime benchmarks such as EffiBench-X [9] omit energy, and language rankings concern human-written code [10], [11]. No study crosses languages with heterogeneous platforms and models their interactions (Table I).

III. Study Design

a) Design.: LLM, language, and platform are fully crossed; tasks are a random factor, and each task–LLM–language cell is sampled five times ($3 \times 3 \times 14 \times 5 = 630$ programs). We measure energy, time, peak memory, and correctness.

b) LLMs.: We use the flagship models of Anthropic, OpenAI, and Google through their official coding CLIs (Table II), so *LLM* denotes model and tool jointly. Requests run non-interactively in an empty directory with tools disabled or sandboxed and effort *medium*. The same zero-shot prompt (Figure 4) never mentions efficiency; code is used unrepaired.

c) Tasks.: Ten EffiBench-X tasks [9] (one Medium and one Hard per category, drawn randomly) leave the algorithm free; four Computer Language Benchmarks Game (CLBG) tasks prescribe it (Table III). One large input per task is solved R times in-process or sized by N, calibrated so that C++ runs about 1 s on the laptop and no reference exceeds 10 s.

d) Platforms and measurement.: Identical language versions run natively (Table IV). Energy comes from RAPL [12], Apple's `powermetrics` estimates, or the Raspberry Pi's PMIC; as their scopes differ, platforms are compared only relatively. On Linux we fix the governor at `performance` and disable turbo; all runs block suspend and record idle power every 30 min.

e) Procedure and analysis.: On every platform, a program must pass 30 test cases (CLBG: reference output) and match the reference on the large input. Correct programs run three times in random order, including start-up. We plan mixed-effects models on log energy with all interactions and random intercepts for task and program, Kendall's W for rank stability, and Spearman correlations (**RQ5**).

IV. Results

a) Generation.: All 630 requests returned one code block without tool use. Claude Code answered fastest, Antigravity CLI slowest (Table V); times include network latency.

b) Measurements.: Of 630 programs, 629 were correct on the laptop and 628 on the Mac, where one Java program printed decimals in the system's German locale. Repetitions varied by 2.6 % (laptop) and 12 % (Mac; median coefficient of variation). Generated Python consumed 11 (Mac: 16) times as much energy as C++ (geometric mean over tasks), Java 2.5 (2.6) times (Figure 1). Single algorithmic choices dominate the LLM comparison (Figure 2): Claude's C++ solutions for EB02 used 150–300 times the others' energy. Relative to the

TABLE I
CLOSEST STUDIES (EXT.: EXTERNAL POWER METER).

Study	LLMs	Languages	Platforms	Measurement	Interact.
Cursaru et al. [8]	1	C++, JS, Py	RPi	Ext.	[TBD]
Cappendijk et al. [6]	5	Py	1 (x86)	RAPL	no
Islam et al. [2]	20	Py	1 (x86 server)	RAPL	no
Solovyeva et al. [1]	3	Py, Java, C++	Mac (M3), PC (x86)	powermetrics, RAPL	no
Apsan et al. [3]	6	Py	Server, PC, RPi	EnergiBridge, Ext.	partly[a]
Rekhi et al. [4]	10	Py, C++	1 (x86 cloud)	RAPL	no
This study	3[b]	Py, Java, C++	Laptop, Server, Mac, RPi	RAPL, powermetrics, PMIC	yes

[a] LLM × prompt interaction tested within each platform; no interaction with platform. [b] Accessed via the vendors' official coding CLIs.

TABLE II
LLMS AND THE CLIS USED TO ACCESS THEM.

Vendor	CLI	Model	Identifier
Anthropic	Claude Code	Claude Opus 5.5	`claude-opus-5-5`
OpenAI	Codex CLI	GPT-6.1 Sol	`gpt-6.1-sol`
Google	Antigravity CLI	Gemini 4 Argon	`gemini-4-argon-medium`

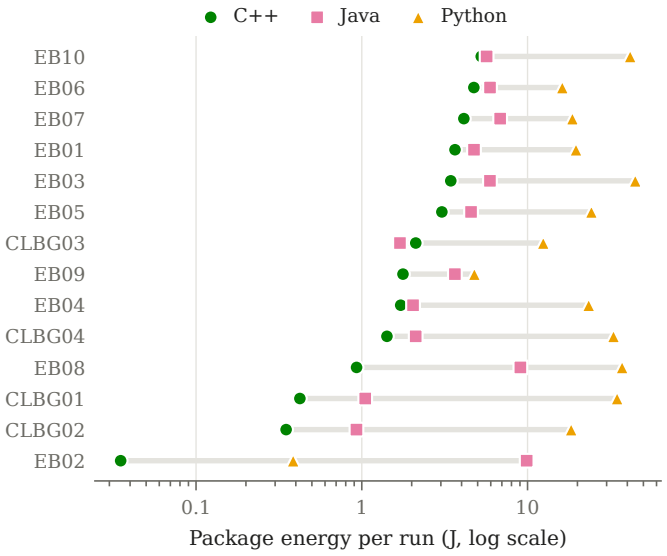


Fig. 1. Median energy per run of generated programs (laptop).

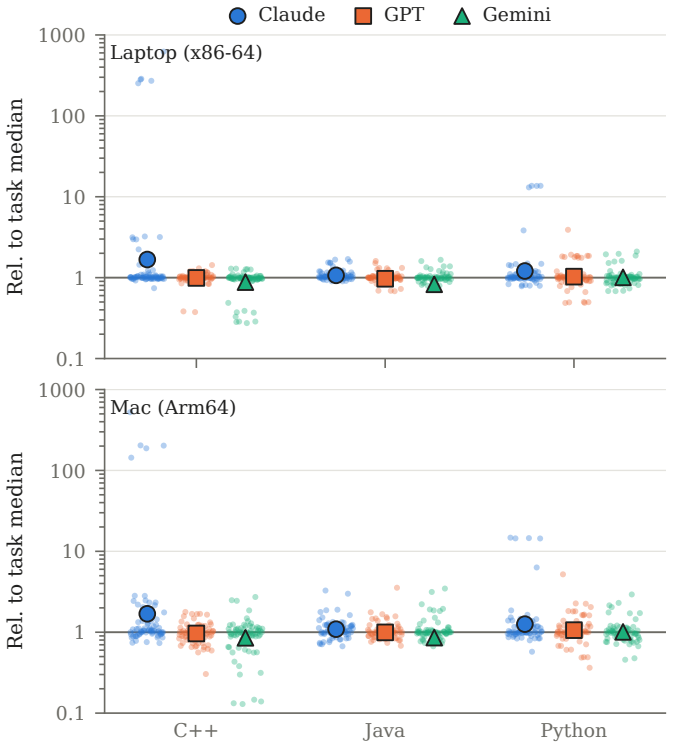


Fig. 2. Program energy relative to its task–language median; large markers: geometric means.

task median, Gemini's programs used 0.90, GPT's 1.00, and Claude's 1.29 on the laptop (Mac: 0.90, 1.01, 1.32). Program energies correlate across platforms ($r = 0.974$, log; Figure 3); energy and time correlate with $r = 0.997$ (laptop) and 0.963 (Mac).

TODO: Server and Raspberry Pi; mixed-effects models for RQ3/RQ4.

V. THREATS TO VALIDITY

RAPL, PMIC, and `powermetrics` (model estimates) capture different scopes, so absolute values are not compared across platforms. The CLIs' hidden system prompts and differently calibrated effort levels make *LLM* a model–tool factor; models and CLIs evolve, so we record versions and dates. All tasks are public and may be memorised. Solving an input repeatedly in one process amortises one-time initialisation, and the 10 s cap leaves some C++ runs below 0.1 s, where start-up weighs more. Results may not generalise to other models, languages, or real-world code.

VI. CONCLUSION

On the laptop and the Mac, language dominates energy, while LLM differences stem from a few algorithmic choices and keep their order across both platforms. Server and Raspberry Pi measurements follow; open-weight models and energy-aware prompting are future work.

TODO: Final conclusion after all platforms; link to replication package.

TABLE III
TASKS (EB: EFFIBENCH-X, WITH LEETCODE RELEASE; CLBG: BENCHMARKS GAME).

ID	Task	Stratum / category	Difficulty	Released
EB01	Maximum Total Damage With Spell Casting	Sorting / searching	Medium	2024-06
EB02	Kth Smallest Amount With Single Denomination Combination	Sorting / searching	Hard	2024-04
EB03	Sum of Digit Differences of All Pairs	Mathematics	Medium	2024-05
EB04	Count the Number of Infection Sequences	Mathematics	Hard	2023-12
EB05	Find Subtree Sizes After Changes	String processing	Medium	2024-10
EB06	Palindrome Rearrangement Queries	String processing	Hard	2023-12
EB07	Maximum Score After Applying Operations on a Tree	Dynamic programming / recursion	Medium	2023-11
EB08	Count the Number of Powerful Integers	Dynamic programming / recursion	Hard	2024-01
EB09	Minimum Number of Coins for Fruits	Data structures	Medium	2023-11
EB10	Longest Common Suffix Queries	Data structures	Hard	2024-03
CLBG01	n-body	Floating-point simulation	–	–
CLBG02	spectral-norm	Floating-point linear algebra	–	–
CLBG03	binary-trees	Memory allocation / recursion	–	–
CLBG04	fannkuch-redux	Permutations / integer arithmetic	–	–

TABLE IV
HARDWARE AND SOFTWARE CONFIGURATION.

	Laptop	Server	Mac	Raspberry Pi 5
CPU / ISA	Intel i7-1185G7 (x86-64)	[TBD] (x86-64)	Apple M4 (Arm64)	Cortex-A76 (Arm64)
RAM	32 GB	16 GB	[TBD]	8 GB
OS / kernel	Ubuntu 24.04, Linux 7.0	[TBD]	macOS 26.3	[TBD]
Python	CPython 3.14 (laptop, Mac: 3.14.10)			
Java	OpenJDK 25 (laptop: 25.0.4.1; Mac: 25.0.5)			
C++	GCC 14, `-O2 -std=c++20` (laptop: 14.2.0; Mac: 14.4.0)			
Energy measurement	RAPL (`powercap`)	RAPL (`powercap`)	`powermetrics` (100 ms)	PMIC ADCs (`vcgencmd`)
Measured scope	CPU package	CPU package + DRAM	CPU+GPU+ANE (model)	board input

TABLE V
GENERATION TIME PER REQUEST IN SECONDS ($n = 210$ PER LLM).

LLM	Mean	SD	Median	EB mean	CLBG mean
Claude	7.1	3.7	5.9	6.1	9.5
GPT	15.1	5.9	13.8	13.6	18.9
Gemini	17.4	13.8	14.9	14.5	24.6

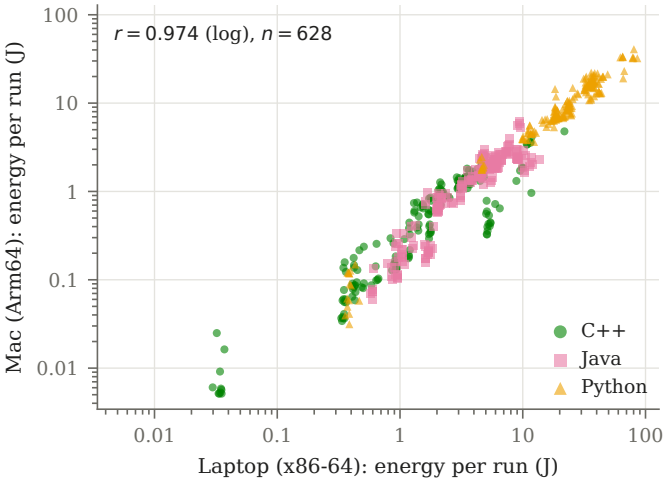


Fig. 3. Energy per program on laptop and Mac (different scopes).

REFERENCES

[1] L. Solovyeva, S. Weidmann, and F. Castor, "AI-powered, but power-hungry? Energy efficiency of LLM-generated code," arXiv preprint arXiv:2502.02412, 2025. [Online]. Available: https://arxiv.org/abs/2502.02412

[2] M. A. Islam, D. V. Jonnala, R. Rekhi, P. Pokharel, S. Cilamkoti, A. Imran, T. Kosar, and B. Turkkan, "Evaluating the energy-efficiency of the code generated by LLMs," arXiv preprint arXiv:2505.20324, 2025. [Online]. Available: https://arxiv.org/abs/2505.20324

[3] R. Apsan, V. Stoico, M. Albonico, R. Dhar, K. Vaidhyanathan, and I. Malavolta, "Generating energy-efficient code via large-language models – where are we now?" in *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering (ICSE)*. ACM, 2026, pp. 426–439, preprint: arXiv:2509.10099.

[4] R. Rekhi, B. Zhang, M. A. Islam, J. K. Pasham, Y. Chitturi, A. Paramesha, I. Valiveti, A. Imran, B. Turkkan, and T. Kosar, "Improving the energy-efficiency of the code generated by LLMs through effective prompting," arXiv preprint arXiv:2610.02571, 2026. [Online]. Available: https://arxiv.org/abs/2610.02571

[5] S. B. Masthan Ali, S. M. Danish, A. Hameed, O. Kirmani, and G. Srivastava, "Energy-efficient code generation using large language models: A systematic literature review," arXiv preprint arXiv:2603.00989, 2026. [Online]. Available: https://arxiv.org/abs/2603.00989

[6] T. Cappendijk, P. de Reus, and A. Oprescu, "An exploration of prompting LLMs to generate energy-efficient code," in *Proceedings of the IEEE/ACM 9th International Workshop on Green and Sustainable Software (GREENS)*. IEEE, 2025, pp. 31–38.

[7] H. Ashraf, S. M. Danish, A. Leivadeas, Y. Otoum, and Z. Sattar, "Energy-aware code generation with LLMs: Benchmarking small vs. large language models for sustainable AI programming," arXiv preprint arXiv:2508.08332, 2025. [Online]. Available: https://arxiv.org/abs/2508.08332

[8] V.-A. Cursaru, L. Duits, J. Milligan, D. Ural, B. Rodriguez Sanchez, V. Stoico, and I. Malavolta, "A controlled experiment on the energy efficiency of the source code generated by Code Llama," arXiv preprint arXiv:2405.03616, 2024. [Online]. Available: https://arxiv.org/abs/2405.03616

[9] Y. Qing, B. Zhu, M. Du, Z. Guo, T. Y. Zhuo, Q. Zhang, J. M. Zhang, H. Cui, S.-M. Yiu, D. Huang, S.-K. Ng, and L. A. Tuan, "EffiBench-X: A multi-language benchmark for measuring efficiency of LLM-generated code," arXiv preprint arXiv:2505.13004, 2025. [Online]. Available: https://arxiv.org/abs/2505.13004

[10] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. Cunha, J. P. Fernandes, and J. Saraiva, "Energy efficiency across programming languages: How do energy, time, and memory relate?" in *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE)*. New York, NY, USA: ACM, 2017, pp. 256–267.

[11] ——, "Ranking programming languages by energy efficiency," *Science of Computer Programming*, vol. 205, p. 102609, 2021.

[12] G. Raffin and D. Trystram, "Dissecting the software-based measurement of CPU energy consumption: A comparative analysis," arXiv preprint arXiv:2401.15985, 2024. [Online]. Available: https://arxiv.org/abs/2401.15985

APPENDIX

Figure 4 shows both prompt templates verbatim; they are included directly from the experiment repository (`experiment/prompts/`). Placeholders in braces are filled per task and language: `{language}` is *Python*, *Java*, or *C++*; `{language_note}` is given in Table VI; `{problem}` is the problem statement (EffiBench-X) or the task specification (CLBG); `{starter}` is the EffiBench-X code skeleton for the respective language. Figure 5 shows a fully rendered prompt.

TABLE VI
LANGUAGE-SPECIFIC LINE `{LANGUAGE_NOTE}`.

Language	Text
Python	Target Python 3.14 (CPython).
Java	Target Java 25 (OpenJDK). *Program template only:* The public class must be named `Main`.
C++	Target C++20. The code is compiled with `g++ -O2 -std=c++20`.

(a) Function template (EffiBench-X tasks)

```
Solve the following programming problem in {language}.

Complete the code skeleton given below. Your code will be combined with a test harness
that reads the input, calls your method, and prints the result. Therefore, do not read
from standard input, do not write to standard output, and do not add a main function.

Requirements:
- Return the complete `Solution` class, including any import or include statements it needs.
- Use only the {language} standard library.
- Do not use multiple threads or processes.
- {language_note}
- Respond with exactly one fenced code block containing the code, and nothing else.

Problem:

{problem}

Code skeleton:

```{fence}
{starter}
```
```

(b) Program template (CLBG tasks)

```
Write a complete {language} program that solves the following task.

Requirements:
- The program reads a single integer N from its first command-line argument.
- It writes its result to standard output, exactly in the format specified below.
- Use only the {language} standard library.
- Do not use multiple threads or processes.
- {language_note}
- Respond with exactly one fenced code block containing the code, and nothing else.

Task:

{problem}
```

Fig. 4. Prompt templates used for all LLMs.

````
Solve the following programming problem in C++.

Complete the code skeleton given below. Your code will be combined with a test harness
that reads the input, calls your method, and prints the result. Therefore, do not read
from standard input, do not write to standard output, and do not add a main function.

Requirements:
- Return the complete `Solution` class, including any import or include statements it needs.
- Use only the C++ standard library.
- Do not use multiple threads or processes.
- Target C++20. The code is compiled with `g++ -O2 -std=c++20`.
- Respond with exactly one fenced code block containing the code, and nothing else.

Problem:

You are given an array `nums` consisting of **positive** integers where all integers have the **same** number of digits.

The **digit difference** between two integers is the *count* of different digits that are in the **same** position in
↪  the two integers.

Return the **sum** of the **digit differences** between **all** pairs of integers in `nums`.

**Example 1:**

**Input:** nums = [13,23,12]

**Output:** 4

**Explanation:**
We have the following:
- The digit difference between **1**3 and **2**3 is 1.
- The digit difference between 1**3** and 1**2** is 1.
- The digit difference between **23** and **12** is 2.
So the total sum of digit differences between all pairs of integers is `1 + 1 + 2 = 4`.

**Example 2:**

**Input:** nums = [10,10,10,10]

**Output:** 0

**Explanation:**
All the integers in the array are the same. So the total sum of digit differences between all pairs of integers will be
↪  0.

**Constraints:**

- `2 <= nums.length <= 10^5`
- `1 <= nums[i] < 10^9`
- All integers in `nums` have the same number of digits.

Code skeleton:

```cpp
class Solution {
public:
    long long sumDigitDifferences(vector<int>& nums) {

    }
};
```
````

Fig. 5. Fully rendered prompt for task EB03 (Sum of Digit Differences of All Pairs) in C++.

OPEN TODOS

TODO: Final results after all platforms. 1
TODO: Server and Raspberry Pi; mixed-effects models for RQ3/RQ4. 2
TODO: Final conclusion after all platforms; link to replication package. 2